

# Natural Language Processing (NLP) and BERT - Lab 5

---

## Lab Overview

- Explain the main idea of Natural Language Processing (NLP), why it matters, and how it converts human language into machine-usable representations.
- Explain the BERT model in detail, including bidirectional context, encoder-only transformer design, masked language modeling, next sentence prediction, and fine-tuning.
- Walk through the uploaded Jupyter notebook that implements a simple BERT-based text classification pipeline on the BBC News dataset.
- Describe how the notebook loads data, encodes labels, tokenizes text, builds a BERT classifier, trains it, evaluates it, plots metrics, and predicts the class of a custom sentence.

## Contents

|                                                                                       |    |
|---------------------------------------------------------------------------------------|----|
| Lab Overview .....                                                                    | 1  |
| Natural Language Processing (NLP) .....                                               | 4  |
| How Natural Language Processing Works .....                                           | 4  |
| 1. Text or Speech Input .....                                                         | 4  |
| 2. Pre-processing .....                                                               | 4  |
| 3. Language Analysis .....                                                            | 4  |
| 4. Text Representation and Embeddings .....                                           | 4  |
| 5. Model Training .....                                                               | 5  |
| 6. Output Generation .....                                                            | 5  |
| Common NLP Tasks and Applications .....                                               | 5  |
| BERT Model .....                                                                      | 5  |
| Why BERT Was Important .....                                                          | 5  |
| Pre-training and Fine-tuning in BERT .....                                            | 5  |
| Workflow of BERT .....                                                                | 5  |
| Masked Language Model (MLM) .....                                                     | 6  |
| Next Sentence Prediction (NSP) .....                                                  | 6  |
| Why MLM and NSP Are Trained Together .....                                            | 6  |
| BERT in This Notebook .....                                                           | 6  |
| Notebook Code (Explained) .....                                                       | 7  |
| Code Cell 1: Environment Setup (Install Packages) (Notebook cell index: 2) .....      | 7  |
| Code Cell 2: Imports and Random Seed (Notebook cell index: 4) .....                   | 7  |
| Code Cell 3: Load the Dataset (Notebook cell index: 6) .....                          | 9  |
| Code Cell 4: Inspect the Dataset (Notebook cell index: 8) .....                       | 10 |
| Code Cell 5: Plot Class Distribution (Notebook cell index: 9) .....                   | 10 |
| Code Cell 6: Prepare Labels and Split the Dataset (Notebook cell index: 11) .....     | 10 |
| Code Cell 7: Tokenization with BERT (Notebook cell index: 13) .....                   | 12 |
| Code Cell 8: Create a PyTorch Dataset and Dataloaders (Notebook cell index: 15) ..... | 13 |
| Code Cell 9: Build the BERT Classification Model (Notebook cell index: 17) .....      | 14 |
| Code Cell 10: Configure Training (Notebook cell index: 19) .....                      | 15 |
| Code Cell 11: Define the run_epoch Helper Function (Notebook cell index: 21) .....    | 16 |

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| Code Cell 12: Train the Model (Notebook cell index: 23) .....                       | 18 |
| Code Cell 13: Plot Training and Validation Loss (Notebook cell index: 25) .....     | 19 |
| Code Cell 14: Plot Training and Validation Accuracy (Notebook cell index: 26) ..... | 20 |
| Code Cell 15: Plot Training and Validation F1-score (Notebook cell index: 27) ..... | 21 |
| Code Cell 16: Evaluate on the Test Set (Notebook cell index: 29) .....              | 21 |
| Code Cell 17: Print the Classification Report (Notebook cell index: 30) .....       | 22 |
| Code Cell 18: Compute and Plot the Confusion Matrix (Notebook cell index: 32) ..... | 22 |
| Code Cell 19: Predict a Custom Text Sample (Notebook cell index: 34) .....          | 23 |
| References .....                                                                    | 24 |

## Natural Language Processing (NLP)

Natural Language Processing (NLP) is the area of artificial intelligence that helps computers understand, interpret, and produce human language. In practice, NLP treats language as data and builds computational models that can analyze structure, meaning, and context in both written and spoken communication.

A useful introductory example is a sentence such as “Ravi is happy with the new phone.” An NLP system can identify Ravi as a person, phone as an object, the sentiment as positive, and the topic as a product review. This shows that NLP is concerned not only with words themselves, but also with relationships, intent, and contextual meaning.

### How Natural Language Processing Works

The referenced GeeksforGeeks introduction presents NLP as a multi-stage pipeline. Although real systems vary, the overall flow usually begins with acquiring language data, then cleaning it, representing it numerically, learning from it, and finally generating a useful output.

#### 1. Text or Speech Input

The process begins with language input. If the source is written text, the system receives the text directly. If the source is spoken language, speech-recognition software converts the audio into text before the NLP pipeline continues.

#### 2. Pre-processing

Pre-processing prepares raw language for analysis. Typical operations include removing punctuation or noisy symbols, splitting text into smaller units such as words or tokens, converting letters to lowercase, removing common stop words, and reducing inflected words to a simpler base form. The purpose is to standardize the input and reduce unnecessary variation.

#### 3. Language Analysis

After cleaning, the system analyzes language structure and meaning. This includes tasks such as part-of-speech recognition, understanding relationships between words, resolving context, detecting entities like names or places, and estimating sentiment. At this stage, the system begins interpreting the linguistic content rather than merely storing text.

#### 4. Text Representation and Embeddings

Because machines operate on numbers rather than raw words, text must be represented numerically. Older NLP methods relied on sparse count-based features such as bag-of-words or TF-IDF. Modern NLP often uses dense embeddings, where words or tokens are mapped to vectors that capture semantic similarity and usage. Contextual embeddings improve this even further by changing a word representation according to the surrounding sentence.

## 5. Model Training

Once the text has been encoded numerically, a learning algorithm can be trained. Traditional NLP often used manually engineered features with statistical machine learning, while modern NLP widely uses neural networks and large pre-trained models. These pre-trained models can later be adapted to specific tasks through fine-tuning.

## 6. Output Generation

The final stage produces an output. Depending on the application, that output could be a class label, a translated sentence, a summary, a chatbot reply, a prediction, or a spoken response.

## Common NLP Tasks and Applications

The GeeksforGeeks article highlights common NLP tasks such as text classification, sentiment analysis, machine translation, named entity recognition, text summarization, and question answering. In practical systems, NLP appears in voice assistants, customer-support chatbots, spam filters, auto-correct systems, translation tools, sentiment-monitoring platforms, and document search engines.

## BERT Model

BERT stands for Bidirectional Encoder Representations from Transformers. It is a transformer-based model designed mainly for language understanding in NLP. The cited GeeksforGeeks explanation emphasizes that BERT uses an encoder-only architecture, meaning it focuses on understanding input sequences rather than generating output sequences with a decoder.

One of BERT's most important innovations is its bidirectional view of context. Older language models often processed text in only one direction, which limited the context available for interpreting a word. BERT instead learns from both the left and right context at the same time, which makes the resulting word representations much richer and more context-aware.

## Why BERT Was Important

BERT represented a major shift in NLP because it combined transformer encoders with large-scale pre-training on unlabeled text. This meant that a single base model could first learn a broad understanding of language and then be fine-tuned for many downstream tasks with comparatively small labeled datasets.

## Pre-training and Fine-tuning in BERT

During pre-training, BERT is exposed to very large amounts of unlabeled text. The goal is not to solve one narrow task, but to learn contextual embeddings and sentence-level relationships that are generally useful. After that, the pre-trained model can be fine-tuned on labeled data for a specific task such as text classification, question answering, or sentence-pair prediction.

## Workflow of BERT

In a BERT workflow, text is tokenized and embedded, then passed through transformer encoder layers. The encoder produces contextualized vectors for the input tokens. These vectors can then feed a task-specific

classification head or another downstream component. This is why BERT is often described as a strong reusable language-understanding backbone.

### Masked Language Model (MLM)

One of BERT's two classic pre-training tasks is the Masked Language Model. In this approach, some words in the input are hidden, and the model must predict the missing words from the surrounding context. Because the prediction depends on both sides of the masked word, BERT learns truly bidirectional context instead of reading the sentence in only one direction.

### Next Sentence Prediction (NSP)

The second pre-training task discussed in the article is Next Sentence Prediction. In this setup, BERT receives two sentences and predicts whether the second sentence actually follows the first one in the original text. This helps the model learn sentence-to-sentence relationships and discourse flow, not just token-level context inside one sentence.

### Why MLM and NSP Are Trained Together

The article explains that combining Masked Language Modeling with Next Sentence Prediction gives BERT a more complete understanding of language. MLM teaches within-sentence context, while NSP teaches inter-sentence relationships. Together they produce a stronger language model for downstream use.

### BERT in This Notebook

The uploaded notebook demonstrates a standard fine-tuning workflow. It loads the pre-trained bert-base-uncased checkpoint, tokenizes text with the matching tokenizer, trains a sequence-classification head on the BBC News dataset, evaluates the model, plots the learning curves, and finally predicts the class of a custom sentence.

## Notebook Code (Explained)

Notes before running the notebook:

- Some paths in the notebook are Windows-specific and may need to be edited before the file is run on another computer.
- The notebook depends on PyTorch, Transformers, scikit-learn, pandas, matplotlib, seaborn, and tqdm.
- The chosen checkpoint is bert-base-uncased, so the tokenizer and the model must both match this checkpoint.
- The notebook uses a train/validation/test split and reports loss, accuracy, weighted F1-score, a classification report, a confusion matrix, and a custom sample prediction.

### Code Cell 1: Environment Setup (Install Packages) (Notebook cell index: 2)

This cell lists the optional installation command that can be used when the required packages are not already available in the environment.

```
# Run this cell only if needed
# %pip install -q torch transformers scikit-learn pandas matplotlib seaborn tqdm
```

Line-by-line explanation:

**Line 1:** This comment is a note for the reader; it documents what the next command is for and is not executed by Python.

**Line 2:** This comment is a note for the reader; it documents what the next command is for and is not executed by Python.

### Code Cell 2: Imports and Random Seed (Notebook cell index: 4)

This cell imports all the required libraries, suppresses non-essential warnings, fixes randomness for reproducibility, and selects the device on which training will run.

```
import os
import random
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm.auto import tqdm

import torch
from torch.utils.data import Dataset, DataLoader

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix, f1_score, accuracy_score
```

```
from torch.optim import AdamW
from transformers import AutoTokenizer, AutoModelForSequenceClassification,
get_linear_schedule_with_warmup

warnings.filterwarnings("ignore")

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("PyTorch version:", torch.__version__)
print("Device:", device)
```

Line-by-line explanation:

**Line 1:** This imports os so the notebook can use the functions, classes, or variables provided by that package.

**Line 2:** This imports random so the notebook can use the functions, classes, or variables provided by that package.

**Line 3:** This imports warnings so the notebook can use the functions, classes, or variables provided by that package.

**Line 4:** This imports Path specifically from pathlib, which makes those tools directly available without writing the full module path each time.

**Line 6:** This imports numpy as np so the notebook can use the functions, classes, or variables provided by that package.

**Line 7:** This imports pandas as pd so the notebook can use the functions, classes, or variables provided by that package.

**Line 8:** This imports matplotlib.pyplot as plt so the notebook can use the functions, classes, or variables provided by that package.

**Line 9:** This imports seaborn as sns so the notebook can use the functions, classes, or variables provided by that package.

**Line 10:** This imports tqdm specifically from tqdm.auto, which makes those tools directly available without writing the full module path each time.

**Line 12:** This imports torch so the notebook can use the functions, classes, or variables provided by that package.

**Line 13:** This imports Dataset, DataLoader specifically from torch.utils.data, which makes those tools directly available without writing the full module path each time.



**Line 15:** This imports `train_test_split` specifically from `sklearn.model_selection`, which makes those tools directly available without writing the full module path each time.

**Line 16:** This imports `LabelEncoder` specifically from `sklearn.preprocessing`, which makes those tools directly available without writing the full module path each time.

**Line 17:** This imports `classification_report`, `confusion_matrix`, `f1_score`, `accuracy_score` specifically from `sklearn.metrics`, which makes those tools directly available without writing the full module path each time.

**Line 19:** This imports `AdamW` specifically from `torch.optim`, which makes those tools directly available without writing the full module path each time.

**Line 20:** This imports `AutoTokenizer`, `AutoModelForSequenceClassification`, `get_linear_schedule_with_warmup` specifically from `transformers`, which makes those tools directly available without writing the full module path each time.

**Line 22:** This suppresses warning messages so the notebook output stays cleaner and easier to read during the lab.

**Line 24:** This stores a fixed random seed value so data splitting and other randomized operations can be reproduced later.

**Line 25:** This applies the fixed seed to Python's built-in random module for reproducibility.

**Line 26:** This applies the same seed to NumPy's random generator so NumPy-based randomness is repeatable.

**Line 27:** This applies the seed to PyTorch on the CPU so model initialization and other PyTorch randomness remain consistent.

**Line 28:** This line starts a conditional block and checks whether `torch.cuda.is_available()` is true before executing the indented code below it.

**Line 29:** This applies the same seed to all available CUDA devices so GPU-based randomness is controlled as well.

**Line 31:** This chooses the execution device dynamically: it uses a CUDA GPU when one is available, otherwise it falls back to the CPU.

**Line 32:** This prints the installed PyTorch version so you can verify which deep-learning library version the notebook is using.

**Line 33:** This prints the chosen computation device so you can confirm whether the notebook is using the GPU or CPU.

### Code Cell 3: Load the Dataset (Notebook cell index: 6)

This cell reads the BBC text dataset from a CSV file and previews the first rows.

```
df = pd.read_csv(r"C:\Users\Lenovo\OneDrive\Desktop\bbc-text.csv")
df.head()
```

Line-by-line explanation:

**Line 1:** This reads the CSV file from disk into a pandas DataFrame so the text dataset can be inspected and processed.

**Line 2:** This displays the first few rows of the DataFrame so you can quickly inspect whether the dataset loaded correctly.

#### Code Cell 4: Inspect the Dataset (Notebook cell index: 8)

This cell checks the size, columns, missing values, and class distribution of the dataset.

```
print("Dataset shape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nMissing values:\n", df.isnull().sum())
print("\nClass distribution:\n", df["category"].value_counts())
```

Line-by-line explanation:

**Line 1:** This prints the number of rows and columns in the dataset so you can confirm the overall dataset size.

**Line 2:** This prints the column names, which helps confirm the structure of the loaded CSV file.

**Line 3:** This checks each column for missing values and prints the result so you can verify data completeness before training.

**Line 4:** This counts how many examples belong to each category so you can inspect class balance in the dataset.

#### Code Cell 5: Plot Class Distribution (Notebook cell index: 9)

This cell visualizes how many samples exist in each class using a count plot.

```
plt.figure(figsize=(8, 4))
sns.countplot(data=df, x="category", order=df["category"].value_counts().index)
plt.title("Class Distribution")
plt.xticks(rotation=30)
plt.show()
```

Line-by-line explanation:

**Line 1:** This opens a new Matplotlib figure and sets its size so the next plot is drawn on a fresh canvas with readable dimensions.

**Line 2:** This draws a Seaborn count plot showing how many samples belong to each category in the dataset.

**Line 3:** This sets the title of the current plot so the figure is easier to interpret.

**Line 4:** This rotates the x-axis labels so long class names do not overlap and remain readable.

**Line 5:** This renders the current figure in the notebook output so the user can actually see the plot.

#### Code Cell 6: Prepare Labels and Split the Dataset (Notebook cell index: 11)

This cell encodes text labels as integers, prepares the text and target arrays, and creates training, validation, and test splits.

```
label_encoder = LabelEncoder()
df["label"] = label_encoder.fit_transform(df["category"])

label_names = list(label_encoder.classes_)
num_labels = len(label_names)

print("Classes:", label_names)
print("Number of classes:", num_labels)

X = df["text"].astype(str).tolist()
y = df["label"].values

# Train: 70%, Validation: 15%, Test: 15%
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=SEED, stratify=y
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=SEED, stratify=y_temp
)

print("Train size:", len(X_train))
print("Validation size:", len(X_val))
print("Test size:", len(X_test))
```

Line-by-line explanation:

**Line 1:** This creates a label encoder object that will convert text class names into numeric class IDs required by the model.

**Line 2:** This learns a mapping from category names to integers and writes those encoded labels into a new DataFrame column.

**Line 4:** This saves the original class names, in the exact order learned by the encoder, for later reporting and interpretation.

**Line 5:** This counts how many distinct classes exist so the BERT classifier's output layer can be created with the correct size.

**Line 7:** This prints the ordered list of class names so you know how numeric labels map back to human-readable categories.

**Line 8:** This prints the total number of distinct classes, which should match the classifier output size.

**Line 10:** This extracts the news article text column, forces each entry to string format, and converts the result into a standard Python list.

**Line 11:** This extracts the encoded class labels as an array that can be used during dataset splitting and model training.

**Line 13:** This comment is a note for the reader; it documents what the next command is for and is not executed by Python.

**Line 14:** This starts the first dataset split, separating the full dataset into a large training portion and a temporary remainder.

**Line 15:** This line passes the texts and labels into the split function, reserves 30% of the data for later validation/testing, uses the fixed seed, and preserves class proportions with stratification.

**Line 16:** This line closes the previous multi-line statement so Python knows the command is complete.

**Line 18:** This starts the second split, dividing the temporary subset into validation data and test data.

**Line 19:** This line splits the temporary subset into two equal halves so validation and test sets each receive 15% of the original dataset.

**Line 20:** This line closes the previous multi-line statement so Python knows the command is complete.

**Line 22:** This prints the number of training samples after splitting.

**Line 23:** This prints the number of validation samples after splitting.

**Line 24:** This prints the number of test samples after splitting.

### Code Cell 7: Tokenization with BERT (Notebook cell index: 13)

This cell selects the BERT checkpoint, loads its tokenizer, and tokenizes the training, validation, and test texts.

```
MODEL_NAME = "bert-base-uncased"
MAX_LEN = 128

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

train_encodings = tokenizer(X_train, truncation=True, padding=True, max_length=MAX_LEN)
val_encodings = tokenizer(X_val, truncation=True, padding=True, max_length=MAX_LEN)
test_encodings = tokenizer(X_test, truncation=True, padding=True, max_length=MAX_LEN)

print("Tokenization complete.")
```

Line-by-line explanation:

**Line 1:** This selects the pre-trained checkpoint to use; bert-base-uncased is a standard English BERT model that expects lowercase text.

**Line 2:** This sets the maximum token length for each text sample so all encoded inputs have a consistent size.

**Line 4:** This loads the tokenizer that matches the chosen BERT checkpoint so raw text can be converted into model-ready tokens.

**Line 6:** This starts tokenizing the training texts, converting them into token IDs and attention masks that BERT can process.

**Line 7:** This starts tokenizing the validation texts using the same tokenizer and settings applied to the training data.

**Line 8:** This starts tokenizing the test texts so the held-out evaluation data uses the same representation as the training data.

**Line 10:** This prints a short confirmation message so the user knows the encoding step finished successfully.

### Code Cell 8: Create a PyTorch Dataset and Dataloaders (Notebook cell index: 15)

This cell wraps the tokenized data in a custom PyTorch dataset class and creates dataloaders for batching.

```
class BBCDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item["labels"] = torch.tensor(int(self.labels[idx]), dtype=torch.long)
        return item

train_dataset = BBCDataset(train_encodings, y_train)
val_dataset = BBCDataset(val_encodings, y_val)
test_dataset = BBCDataset(test_encodings, y_test)

BATCH_SIZE = 8

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print("Dataloaders ready.")
```

Line-by-line explanation:

**Line 1:** This line defines a custom class named `BBCDataset`, which groups related data and methods into one reusable object.

**Line 2:** This line defines a function named `__init__`; the indented lines below describe the steps the function will perform whenever it is called.

**Line 3:** This saves the tokenized inputs inside the dataset object so they can be accessed later.

**Line 4:** This saves the class labels inside the dataset object so each text sample can be paired with its target class.

**Line 6:** This line defines a function named `__len__`; the indented lines below describe the steps the function will perform whenever it is called.

**Line 7:** This line sends the computed values back to the place where the function was called so they can be used later in the notebook.

**Line 9:** This line defines a function named `__getitem__`; the indented lines below describe the steps the function will perform whenever it is called.

**Line 10:** This builds a dictionary for one sample by converting the tokenized values at index `idx` into PyTorch tensors.

**Line 11:** This adds the target label for the current sample as a long integer tensor, which is the format expected for classification loss.

**Line 12:** This line sends the computed values back to the place where the function was called so they can be used later in the notebook.

**Line 14:** This creates the training dataset object by pairing the tokenized training texts with their labels.

**Line 15:** This creates the validation dataset object in the same way for validation samples.

**Line 16:** This creates the test dataset object for the final held-out evaluation.

**Line 18:** This sets how many samples will be processed together in one batch during training or evaluation.

**Line 20:** This creates a dataloader for the training set so data is delivered to the model in shuffled mini-batches.

**Line 21:** This creates a dataloader for the validation set; shuffling is disabled because evaluation does not require randomized order.

**Line 22:** This creates a dataloader for the test set so the final evaluation can be performed batch by batch.

**Line 24:** This prints a confirmation message to show that the dataset wrappers and dataloaders were created successfully.

### Code Cell 9: Build the BERT Classification Model (Notebook cell index: 17)

This cell loads a pre-trained BERT classifier and moves it to the selected device.

```
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=num_labels
)
model.to(device)

print("Model loaded successfully.")
```

Line-by-line explanation:

**Line 1:** This starts loading a pre-trained BERT model that already has a sequence-classification head suitable for text classification tasks.

**Line 2:** This passes the chosen pre-trained checkpoint name into the model-loading function.

**Line 3:** This tells the classifier head how many output classes it must predict.

**Line 4:** This line closes the previous multi-line statement so Python knows the command is complete.

**Line 5:** This moves the model to the selected device, such as the GPU if available or the CPU otherwise.

**Line 7:** This prints a confirmation message after the pre-trained BERT classifier has been loaded into memory.

### Code Cell 10: Configure Training (Notebook cell index: 19)

This cell defines the training hyperparameters, optimizer, scheduler, and metric-history dictionary.

```
EPOCHS = 3
LEARNING_RATE = 2e-5

optimizer = AdamW(model.parameters(), lr=LEARNING_RATE)

total_steps = len(train_loader) * EPOCHS
scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=0,
    num_training_steps=total_steps
)

history = {
    "train_loss": [],
    "val_loss": [],
    "train_acc": [],
    "val_acc": [],
    "train_f1": [],
    "val_f1": []
}
```

Line-by-line explanation:

**Line 1:** This sets how many full passes over the training data the notebook will perform.

**Line 2:** This defines the optimizer's learning rate, controlling how large each weight update will be.

**Line 4:** This creates an AdamW optimizer that will update all trainable model parameters using the specified learning rate.

**Line 6:** This calculates the total number of optimization steps that will occur across all training epochs.

**Line 7:** This starts the creation of a learning-rate scheduler that will gradually change the learning rate over training.

**Line 8:** This passes the optimizer object into the scheduler so the scheduler can control that optimizer's learning rate.

**Line 9:** This sets the number of warm-up steps to zero, meaning the learning rate schedule begins its linear behavior immediately.

**Line 10:** This tells the scheduler how many total training steps it should plan for across the whole training run.

**Line 11:** This line closes the previous multi-line statement so Python knows the command is complete.

**Line 13:** This starts a dictionary that will store the training and validation metrics collected after each epoch.

**Line 14:** This creates an empty list for storing the training loss after each epoch.

**Line 15:** This creates an empty list for storing the validation loss after each epoch.

**Line 16:** This creates an empty list for storing training accuracy after each epoch.

**Line 17:** This creates an empty list for storing validation accuracy after each epoch.

**Line 18:** This creates an empty list for storing the training F1-score after each epoch.

**Line 19:** This creates an empty list for storing the validation F1-score after each epoch.

**Line 20:** This line closes the previous multi-line statement so Python knows the command is complete.

### Code Cell 11: Define the `run_epoch` Helper Function (Notebook cell index: 21)

This cell defines the helper function that performs one full pass through a dataloader in either training mode or evaluation mode.

```
def run_epoch(model, data_loader, optimizer=None, scheduler=None, train_mode=True):
    if train_mode:
        model.train()
    else:
        model.eval()

    losses = []
    all_preds = []
    all_labels = []

    for batch in tqdm(data_loader, leave=False):
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        if train_mode:
            optimizer.zero_grad()

        with torch.set_grad_enabled(train_mode):
            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels
            )
            loss = outputs.loss
            logits = outputs.logits

            if train_mode:
                loss.backward()
                optimizer.step()
                if scheduler is not None:
                    scheduler.step()

        preds = torch.argmax(logits, dim=1)

        losses.append(loss.item())
        all_preds.extend(preds.detach().cpu().numpy())
        all_labels.extend(labels.detach().cpu().numpy())
```



```
avg_loss = np.mean(losses)
acc = accuracy_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds, average="weighted")

return avg_loss, acc, f1, all_labels, all_preds
```

Line-by-line explanation:

**Line 1:** This line defines a function named `run_epoch`; the indented lines below describe the steps the function will perform whenever it is called.

**Line 2:** This line starts a conditional block and checks whether `train_mode` is true before executing the indented code below it.

**Line 3:** This switches the model into training mode so layers such as dropout or batch-normalization behave correctly while learning.

**Line 4:** This begins the alternative branch that will run only when the previous condition is false.

**Line 5:** This switches the model into evaluation mode so it behaves deterministically during validation or prediction.

**Line 7:** This creates an empty list that will collect the loss value from each batch so an average epoch loss can be computed later.

**Line 8:** This creates an empty list for storing the predicted class labels from all batches in the epoch.

**Line 9:** This creates an empty list for storing the true class labels from all batches in the epoch.

**Line 11:** This line starts a loop that repeats the indented block for each item produced by the iterable on the right-hand side.

**Line 12:** This extracts the input token IDs from the current batch and moves them to the selected device.

**Line 13:** This extracts the attention mask and moves it to the same device so BERT knows which tokens are real and which are padding.

**Line 14:** This extracts the true class labels for the current batch and moves them to the selected device.

**Line 16:** This line starts a conditional block and checks whether `train_mode` is true before executing the indented code below it.

**Line 17:** This clears any gradients left over from the previous batch so gradient values do not accumulate by mistake.

**Line 19:** This context manager enables gradient tracking only in training mode and disables it during evaluation to save memory and computation.

**Line 20:** This starts a forward pass through the model using the current batch inputs and labels.

**Line 21:** This passes the token ID tensor into BERT as the main textual input.

**Line 22:** This passes the attention mask so the model can ignore padded positions.

**Line 23:** This passes the true labels into the model, allowing Hugging Face to compute the classification loss automatically.

**Line 24:** This line closes the previous multi-line statement so Python knows the command is complete.

**Line 25:** This extracts the scalar loss value returned by the model for the current batch.

**Line 26:** This extracts the raw class scores, called logits, that the model produced before softmax.

**Line 28:** This line starts a conditional block and checks whether `train_mode` is true before executing the indented code below it.

**Line 29:** This performs backpropagation, computing how much each trainable parameter contributed to the current loss.

**Line 30:** This tells the optimizer to update the model weights using the gradients that were just computed.

**Line 31:** This line starts a conditional block and checks whether `scheduler is not None` is true before executing the indented code below it.

**Line 32:** This advances the learning-rate scheduler by one step so the learning rate follows the planned schedule during training.

**Line 34:** This converts the logits into predicted class indices by selecting the highest-scoring class for each sample.

**Line 36:** This stores the numeric loss value for the current batch in the list of batch losses.

**Line 37:** This moves the predictions to the CPU, converts them to NumPy values, and appends them to the running list of predictions.

**Line 38:** This does the same conversion for the true labels so they can later be compared with the predictions.

**Line 40:** This calculates the mean batch loss to obtain one overall loss value for the entire epoch.

**Line 41:** This computes classification accuracy by comparing the true labels with the predicted labels.

**Line 42:** This computes the weighted F1-score, which summarizes precision and recall while taking class frequencies into account.

**Line 44:** This line sends the computed values back to the place where the function was called so they can be used later in the notebook.

### Code Cell 12: Train the Model (Notebook cell index: 23)

This cell performs the main training loop across epochs, validates after each epoch, and stores the metrics.

```
for epoch in range(EPOCHS):
    print(f"Epoch {epoch+1}/{EPOCHS}")

    train_loss, train_acc, train_f1, _, _ = run_epoch(
        model, train_loader, optimizer=optimizer, scheduler=scheduler, train_mode=True
    )

    val_loss, val_acc, val_f1, _, _ = run_epoch(
        model, val_loader, train_mode=False
    )

    history["train_loss"].append(train_loss)
    history["val_loss"].append(val_loss)
    history["train_acc"].append(train_acc)
```

```
history["val_acc"].append(val_acc)
history["train_f1"].append(train_f1)
history["val_f1"].append(val_f1)

print(f"Train Loss: {train_loss:.4f} | Train Acc: {train_acc:.4f} | Train F1: {train_f1:.4f}")
print(f"Val Loss: {val_loss:.4f} | Val Acc: {val_acc:.4f} | Val F1: {val_f1:.4f}")
print("-" * 60)
```

Line-by-line explanation:

**Line 1:** This line starts a loop that repeats the indented block for each item produced by the iterable on the right-hand side.

**Line 2:** This prints the current epoch number in a user-friendly format.

**Line 4:** This calls the helper function in training mode and stores the resulting training loss, accuracy, and F1-score for the epoch.

**Line 5:** This passes the model, training dataloader, optimizer, scheduler, and a training-mode flag into the `run_epoch` function.

**Line 6:** This line closes the previous multi-line statement so Python knows the command is complete.

**Line 8:** This calls the same helper function in evaluation mode on the validation set so the model can be monitored after each epoch.

**Line 9:** This passes the model and validation dataloader to `run_epoch` and disables training behavior such as gradient updates.

**Line 10:** This line closes the previous multi-line statement so Python knows the command is complete.

**Line 12:** This stores the epoch's training loss in the history dictionary for later plotting.

**Line 13:** This stores the epoch's validation loss in the history dictionary.

**Line 14:** This stores the epoch's training accuracy.

**Line 15:** This stores the epoch's validation accuracy.

**Line 16:** This stores the epoch's training F1-score.

**Line 17:** This stores the epoch's validation F1-score.

**Line 19:** This prints a formatted summary of the training metrics for the current epoch.

**Line 20:** This prints a formatted summary of the validation metrics for the current epoch.

**Line 21:** This prints a separator line to make the output from consecutive epochs easier to read.

### Code Cell 13: Plot Training and Validation Loss (Notebook cell index: 25)

This cell plots the training and validation loss over the training epochs.

```
epochs_range = range(1, EPOCHS + 1)

plt.figure(figsize=(6, 4))
plt.plot(epochs_range, history["train_loss"], marker="o", label="Train Loss")
plt.plot(epochs_range, history["val_loss"], marker="o", label="Validation Loss")
plt.title("Training and Validation Loss")
```

```
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
```

Line-by-line explanation:

**Line 1:** This creates the sequence of epoch numbers that will be used on the x-axis of the training plots.

**Line 3:** This opens a new Matplotlib figure and sets its size so the next plot is drawn on a fresh canvas with readable dimensions.

**Line 4:** This plots the training-loss values across epochs.

**Line 5:** This plots the validation-loss values across epochs so they can be compared with the training curve.

**Line 6:** This sets the title of the current plot so the figure is easier to interpret.

**Line 7:** This line performs part of the current operation and contributes to the overall processing step shown in this code cell.

**Line 8:** This labels the y-axis as loss.

**Line 9:** This displays a legend so the plotted curves can be distinguished from one another.

**Line 10:** This adds a background grid to the graph so values are easier to estimate visually.

**Line 11:** This renders the current figure in the notebook output so the user can actually see the plot.

### Code Cell 14: Plot Training and Validation Accuracy (Notebook cell index: 26)

This cell plots the training and validation accuracy over the epochs.

```
plt.figure(figsize=(6, 4))
plt.plot(epochs_range, history["train_acc"], marker="o", label="Train Accuracy")
plt.plot(epochs_range, history["val_acc"], marker="o", label="Validation Accuracy")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
```

Line-by-line explanation:

**Line 1:** This opens a new Matplotlib figure and sets its size so the next plot is drawn on a fresh canvas with readable dimensions.

**Line 2:** This plots the training-accuracy values across epochs.

**Line 3:** This plots the validation-accuracy values across epochs.

**Line 4:** This sets the title of the current plot so the figure is easier to interpret.

**Line 5:** This line performs part of the current operation and contributes to the overall processing step shown in this code cell.

**Line 6:** This labels the y-axis as accuracy.

**Line 7:** This displays a legend so the plotted curves can be distinguished from one another.

**Line 8:** This adds a background grid to the graph so values are easier to estimate visually.

**Line 9:** This renders the current figure in the notebook output so the user can actually see the plot.

### Code Cell 15: Plot Training and Validation F1-score (Notebook cell index: 27)

This cell plots the training and validation weighted F1-score over the epochs.

```
plt.figure(figsize=(6, 4))
plt.plot(epochs_range, history["train_f1"], marker="o", label="Train F1-score")
plt.plot(epochs_range, history["val_f1"], marker="o", label="Validation F1-score")
plt.title("Training and Validation F1-score")
plt.xlabel("Epoch")
plt.ylabel("F1-score")
plt.legend()
plt.grid(True)
plt.show()
```

Line-by-line explanation:

**Line 1:** This opens a new Matplotlib figure and sets its size so the next plot is drawn on a fresh canvas with readable dimensions.

**Line 2:** This plots the training F1-score values across epochs.

**Line 3:** This plots the validation F1-score values across epochs.

**Line 4:** This sets the title of the current plot so the figure is easier to interpret.

**Line 5:** This line performs part of the current operation and contributes to the overall processing step shown in this code cell.

**Line 6:** This labels the y-axis as F1-score.

**Line 7:** This displays a legend so the plotted curves can be distinguished from one another.

**Line 8:** This adds a background grid to the graph so values are easier to estimate visually.

**Line 9:** This renders the current figure in the notebook output so the user can actually see the plot.

### Code Cell 16: Evaluate on the Test Set (Notebook cell index: 29)

This cell evaluates the final model on the unseen test data and prints the resulting metrics.

```
test_loss, test_acc, test_f1, y_true, y_pred = run_epoch(
    model, test_loader, train_mode=False
)

print(f"Test Loss: {test_loss:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")
print(f"Test Weighted F1-score: {test_f1:.4f}")
```

Line-by-line explanation:

**Line 1:** This runs one full evaluation pass over the test set and stores the test metrics together with the true and predicted labels.

**Line 2:** This evaluates the model on the test dataloader with training behavior disabled.

**Line 3:** This line closes the previous multi-line statement so Python knows the command is complete.

**Line 5:** This prints the final average loss on the unseen test set.

**Line 6:** This prints the final classification accuracy on the test set.

**Line 7:** This prints the weighted F1-score on the test set, giving a balanced summary of performance across classes.

### Code Cell 17: Print the Classification Report (Notebook cell index: 30)

This cell prints a detailed classification report for all categories.

```
print(classification_report(y_true, y_pred, target_names=label_names))
```

Line-by-line explanation:

**Line 1:** This prints a full classification report showing precision, recall, and F1-score for each category using the original class names.

### Code Cell 18: Compute and Plot the Confusion Matrix (Notebook cell index: 32)

This cell computes the confusion matrix and displays it as a heatmap.

```
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=label_names, yticklabels=label_names)
plt.title("Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()
```

Line-by-line explanation:

**Line 1:** This computes the confusion matrix, showing how often each true class was predicted as each possible class.

**Line 3:** This opens a new Matplotlib figure and sets its size so the next plot is drawn on a fresh canvas with readable dimensions.

**Line 4:** This starts drawing a heatmap of the confusion matrix, annotating each cell with integer counts and using a blue color palette.

**Line 5:** This labels the heatmap axes with the human-readable class names so the confusion matrix is easy to interpret.

**Line 6:** This sets the title of the current plot so the figure is easier to interpret.

**Line 7:** This labels the x-axis to show that the columns correspond to predicted classes.

**Line 8:** This labels the y-axis to show that the rows correspond to true classes.

**Line 9:** This renders the current figure in the notebook output so the user can actually see the plot.

### Code Cell 19: Predict a Custom Text Sample (Notebook cell index: 34)

This cell defines a reusable prediction function, applies it to a custom sentence, and prints the probability for each class.

```
def predict_text(text, model, tokenizer, label_encoder, max_len=128):
    model.eval()
    encoding = tokenizer(
        text,
        truncation=True,
        padding="max_length",
        max_length=max_len,
        return_tensors="pt"
    )

    input_ids = encoding["input_ids"].to(device)
    attention_mask = encoding["attention_mask"].to(device)

    with torch.no_grad():
        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        probs = torch.softmax(outputs.logits, dim=1)
        pred_label = torch.argmax(probs, dim=1).item()

    predicted_class = label_encoder.inverse_transform([pred_label])[0]
    return predicted_class, probs.cpu().numpy().flatten()

sample_text = "The government announced new economic policies and parliament debated the national budget."
predicted_class, probabilities = predict_text(sample_text, model, tokenizer, label_encoder)

print("Text:", sample_text)
print("Predicted class:", predicted_class)
print("\nClass probabilities:")
for class_name, prob in zip(label_names, probabilities):
    print(f"{class_name}: {prob:.4f}")
```

Line-by-line explanation:

**Line 1:** This line defines a function named `predict_text`; the indented lines below describe the steps the function will perform whenever it is called.

**Line 2:** This switches the model into evaluation mode so it behaves deterministically during validation or prediction.

**Line 3:** This starts tokenizing the input text so it can be passed into the BERT model.

**Line 4:** This passes the raw input sentence into the tokenizer.

**Line 5:** This line continues the argument list or data structure started above, supplying another value needed to complete that statement.

**Line 6:** This pads shorter sequences up to the maximum length so the input tensor has a fixed shape.

**Line 7:** This line continues the argument list or data structure started above, supplying another value needed to complete that statement.

**Line 8:** This tells the tokenizer to return PyTorch tensors directly instead of regular Python lists.

**Line 9:** This line closes the previous multi-line statement so Python knows the command is complete.

**Line 11:** This extracts the token IDs from the tokenizer output and moves them to the selected device.

**Line 12:** This extracts the attention mask from the tokenizer output and moves it to the same device.

**Line 14:** This disables gradient calculations because the function is only performing inference, not training.

**Line 15:** This starts a forward pass through the model using the current batch inputs and labels.

**Line 16:** This converts the raw logits into probabilities so each class receives a normalized score between 0 and 1.

**Line 17:** This selects the class index with the highest predicted probability and converts it into a regular Python integer.

**Line 19:** This converts the numeric predicted label back into its original text class name.

**Line 20:** This line sends the computed values back to the place where the function was called so they can be used later in the notebook.

**Line 22:** This defines one example sentence that will be passed to the trained classifier for a final demonstration.

**Line 23:** This calls the prediction function on the sample sentence and stores both the predicted class and the probability scores.

**Line 25:** This prints the original example text so the reader can compare the input with the model's prediction.

**Line 26:** This prints the final class label predicted by the model for the sample sentence.

**Line 27:** This prints a heading before listing the probability assigned to each class.

**Line 28:** This line starts a loop that repeats the indented block for each item produced by the iterable on the right-hand side.

**Line 29:** This prints each class name and its probability rounded to four decimal places.

## References

- GeeksforGeeks, "Introduction to Natural Language Processing (NLP)," last updated 24 February 2026.
- GeeksforGeeks, "BERT Model - NLP," last updated 11 September 2025.
- Uploaded notebook: Advanced\_HCI\_Python\_NLP.ipynb.

### Source URLs:

<https://www.geeksforgeeks.org/nlp/introduction-to-natural-language-processing-nlp/>

<https://www.geeksforgeeks.org/nlp/explanation-of-bert-model-nlp/>